

```
In [1]: from pathlib import Path
import duckdb
import numpy as np
import pandas as pd
con = duckdb.connect("warehouse.duckdb")
```

3. Crear el esquema estrella en DuckDB

```
In [2]: con.execute("""
DROP TABLE IF EXISTS fact_sales;
DROP TABLE IF EXISTS dim_time;
DROP TABLE IF EXISTS dim_item;
DROP TABLE IF EXISTS dim_location;

-- DIMENSIONES – atributos descriptivos y jerarquías
CREATE TABLE dim_time (
time_key INTEGER PRIMARY KEY,
day DATE, month VARCHAR, quarter VARCHAR, year INTEGER );

CREATE TABLE dim_item (
item_key INTEGER PRIMARY KEY,
item_name VARCHAR, brand VARCHAR, category VARCHAR );

CREATE TABLE dim_location (
loc_key INTEGER PRIMARY KEY,
city VARCHAR, state VARCHAR, country VARCHAR );
-- HECHOS – claves foraneas + medidas numericas

CREATE TABLE fact_sales (
time_key INTEGER REFERENCES dim_time(time_key),
item_key INTEGER REFERENCES dim_item(item_key),
loc_key INTEGER REFERENCES dim_location(loc_key),
dollars_sold DECIMAL(12,2),
units_sold INTEGER );
""")
```

Out[2]: <duckdb.DuckDBPyConnection at 0x2ca342f7bb0>

```
In [3]: con.execute("""
INSERT INTO dim_time SELECT * FROM read_csv_auto('data/dim_time.csv');
INSERT INTO dim_item SELECT * FROM read_csv_auto('data/dim_item.csv');
INSERT INTO dim_location SELECT * FROM read_csv_auto('data/dim_location.csv');
INSERT INTO fact_sales SELECT * FROM read_csv_auto('data/fact_sales.csv');
""")
```

Out[3]: <duckdb.DuckDBPyConnection at 0x2ca342f7bb0>

```
In [4]: con.execute("""
DROP VIEW IF EXISTS v_sales;
CREATE VIEW v_sales AS
SELECT t.year, t.quarter, t.month, i.category, i.item_name, l.country, f.dollar
FROM fact_sales f
```

```
JOIN dim_time t USING (time_key)
JOIN dim_item i USING (item_key)
JOIN dim_location l USING (loc_key);
"""
```

Out[4]: <_duckdb.DuckDBPyConnection at 0x2ca342f7bb0>

4. Operaciones OLAP — código ejecutable

Roll-up y drill-down

Roll-up y drill-down son para escoger el nivel de los datos de una cara o dimensión. En este caso indica que tanto detalle va a tener la información que se va a mostrar, Roll up es para menos detalles tomando un nivel más general, mientras que drill-down es tomar más detalle.

Ejemplos: Roll-up: pasar de cities y country en la consulta a solo country. Drill-down: pasar year a year, month y quarter.

```
In [5]: RollUp = con.execute("""
        SELECT country, SUM(dollars_sold), SUM(units_sold) FROM v_sales GROUP BY country
        """).df()
RollUp
```

Out[5]:

	country	sum(dollars_sold)	sum(units_sold)
0	Chile	149523.18	5246.0
1	USA	371309.46	11613.0
2	Mexico	382387.89	14230.0
3	Colombia	184710.89	7807.0

	country	sum(dollars_sold)	sum(units_sold)
0	Chile	149523.18	5246.0
1	USA	371309.46	11613.0
2	Mexico	382387.89	14230.0
3	Colombia	184710.89	7807.0

Resultado: La consulta nos va a otorgar la cantidad de dolares y unidades vendidas agrupados por el país.

```
In [6]: drillDown = con.execute("""
        SELECT year, month, quarter, SUM(dollars_sold), SUM(units_sold) FROM v_sales
        GROUP BY year, month, quarter;
        """).df()
drillDown
```

Out[6]:

	year	month	quarter	sum(dollars_sold)	sum(units_sold)
0	2025	9	3	84327.79	3113.0
1	2025	5	2	85397.28	3121.0
2	2025	8	3	88701.09	3309.0
3	2025	12	4	111041.98	3323.0
4	2025	11	4	97816.87	3018.0
5	2025	6	2	89586.91	3338.0
6	2025	3	1	91386.51	3346.0
7	2025	4	2	91053.99	3330.0
8	2025	2	1	81791.69	3080.0
9	2025	7	3	91463.75	3436.0
10	2025	1	1	83436.45	3163.0
11	2025	10	4	91927.11	3319.0

Resultado: La consulta nos va a otorgar la cantidad de dolares y unidades vendidas agrupadas por año, mes y quarter

Slice & dice

Sirve para obtener los datos de una o varias caras/dimensiones del cubo.

In [7]:

```
Slice_Dice =con.execute("""
    SELECT category, SUM(dollars_sold), SUM(units_sold) FROM v_sales
    WHERE country = 'USA' GROUP BY category;
""").df()
Slice_Dice
```

Out[7]:

	category	sum(dollars_sold)	sum(units_sold)
0	Grocery	114985.81	3601.0
1	Beauty	78809.76	2665.0
2	Sports	65871.35	2004.0
3	Electronics	47506.61	1416.0
4	Home	64135.93	1927.0

Resultado: La consulta nos otorga cuanto se vendio con respecto a las variables (dollars_sold y units_sold) por categorias en el pais de USA

Pivot

Cambia la forma en como se ve el resultado, intercambia las filas por las columnas.

```
In [8]: pivot = con.execute("""
        PIVOT v_sales ON country USING SUM(dollars_sold) GROUP BY category;
        """).df()
pivot
```

```
Out[8]:
```

	category	Chile	Colombia	Mexico	USA
0	Electronics	19559.61	21901.29	51265.67	47506.61
1	Home	25176.05	31758.18	61481.04	64135.93
2	Beauty	32357.55	40182.81	75143.06	78809.76
3	Sports	26312.22	30887.24	71760.33	65871.35
4	Grocery	46117.75	59981.37	122737.79	114985.81

Resultado: Lo que hace es mostrarnos las ventas que hay por paises agrupados por categorias, entonces en las columnas nos mostrara a los paises y en las filas las categorias

5. Materialización con CUBE y ROLLUP

CUBE

```
In [9]: cube_df = con.execute("""
        SELECT year, country, category,
               SUM(dollars_sold) AS dollars_sold,
               SUM(units_sold) AS units_sold
        FROM v_sales
        GROUP BY CUBE (year, country, category)
        """).df()

levels = con.execute("""
        SELECT
            COUNT(DISTINCT year) AS l_year,
            COUNT(DISTINCT country) AS l_country,
            COUNT(DISTINCT category) AS l_category
        FROM v_sales
        """).df()

cube_df
```

Out[9]:

	year	country	category	dollars_sold	units_sold
0	<NA>	NaN	NaN	1087931.42	38896.0
1	2025	NaN	NaN	1087931.42	38896.0
2	2025	USA	NaN	371309.46	11613.0
3	2025	Mexico	NaN	382387.89	14230.0
4	2025	Mexico	Sports	71760.33	2600.0
5	2025	Chile	Beauty	32357.55	1197.0
6	2025	Mexico	Electronics	51265.67	1865.0
7	2025	USA	Beauty	78809.76	2665.0
8	2025	Colombia	Sports	30887.24	1266.0
9	2025	Mexico	Home	61481.04	2217.0
10	2025	Mexico	Grocery	122737.79	4596.0
11	2025	Colombia	Beauty	40182.81	1811.0
12	2025	Colombia	Home	31758.18	1284.0
13	2025	Colombia	Grocery	59981.37	2528.0
14	2025	Colombia	Electronics	21901.29	918.0
15	2025	USA	Grocery	114985.81	3601.0
16	2025	USA	Home	64135.93	1927.0
17	2025	Chile	NaN	149523.18	5246.0
18	2025	NaN	Sports	194831.14	6779.0
19	2025	Chile	Grocery	46117.75	1632.0
20	2025	Chile	Home	25176.05	822.0
21	<NA>	Mexico	NaN	382387.89	14230.0
22	2025	NaN	Beauty	226493.18	8625.0
23	2025	USA	Electronics	47506.61	1416.0
24	2025	Chile	Sports	26312.22	909.0
25	2025	Mexico	Beauty	75143.06	2952.0
26	2025	NaN	Home	182551.20	6250.0
27	2025	NaN	Grocery	343822.72	12357.0
28	2025	NaN	Electronics	140233.18	4885.0
29	2025	USA	Sports	65871.35	2004.0

	year	country	category	dollars_sold	units_sold
30	2025	Chile	Electronics	19559.61	686.0
31	<NA>	USA	NaN	371309.46	11613.0
32	<NA>	Colombia	NaN	184710.89	7807.0
33	<NA>	Chile	NaN	149523.18	5246.0
34	<NA>	Chile	Grocery	46117.75	1632.0
35	<NA>	Chile	Home	25176.05	822.0
36	<NA>	USA	Grocery	114985.81	3601.0
37	<NA>	USA	Home	64135.93	1927.0
38	<NA>	Colombia	Home	31758.18	1284.0
39	<NA>	Colombia	Grocery	59981.37	2528.0
40	<NA>	USA	Beauty	78809.76	2665.0
41	<NA>	Colombia	Sports	30887.24	1266.0
42	<NA>	Mexico	Beauty	75143.06	2952.0
43	<NA>	Chile	Electronics	19559.61	686.0
44	<NA>	Mexico	Home	61481.04	2217.0
45	<NA>	Mexico	Grocery	122737.79	4596.0
46	2025	Colombia	NaN	184710.89	7807.0
47	<NA>	Chile	Beauty	32357.55	1197.0
48	<NA>	USA	Sports	65871.35	2004.0
49	<NA>	Chile	Sports	26312.22	909.0
50	<NA>	USA	Electronics	47506.61	1416.0
51	<NA>	Colombia	Beauty	40182.81	1811.0
52	<NA>	Mexico	Sports	71760.33	2600.0
53	<NA>	Colombia	Electronics	21901.29	918.0
54	<NA>	Mexico	Electronics	51265.67	1865.0
55	<NA>	NaN	Beauty	226493.18	8625.0
56	<NA>	NaN	Home	182551.20	6250.0
57	<NA>	NaN	Grocery	343822.72	12357.0
58	<NA>	NaN	Electronics	140233.18	4885.0
59	<NA>	NaN	Sports	194831.14	6779.0

Verificacion del conteo de CUBE

Para `GROUP BY CUBE(year, country, category)` se cuentan las filas devueltas por la consulta y se comparan contra la formula del material: $(L_{year} + 1) * (L_{country} + 1) * (L_{category} + 1)$. Aqui `L` representa la cantidad de valores distintos en cada dimension usada en el cubo. Si el resultado coincide, significa que el cubo tiene todos los niveles esperados para esas dimensiones.

```
In [10]: cube_rows = len(cube_df)
level_counts = con.execute("""
    SELECT
        COUNT(DISTINCT year) AS l_year,
        COUNT(DISTINCT country) AS l_country,
        COUNT(DISTINCT category) AS l_category
    FROM v_sales
""").df().iloc[0]

expected_cube_rows = int(
    (level_counts["l_year"] + 1)
    * (level_counts["l_country"] + 1)
    * (level_counts["l_category"] + 1)
)

cube_check_df = pd.DataFrame(
    {
        "metrica": [
            "filas_devueltas_por_cube",
            "filas_esperadas_formula",
            "coincide",
        ],
        "valor": [
            cube_rows,
            expected_cube_rows,
            cube_rows == expected_cube_rows,
        ],
    }
)

cube_check_df
```

```
Out[10]:
```

	metrica	valor
0	filas_devueltas_por_cube	60
1	filas_esperadas_formula	60
2	coincide	True

ROLLUP

```
In [11]: rollup_df = con.execute("""
        SELECT year, country, category, SUM(dollars_sold) AS dollars_sold, SUM(units_so
        FROM v_sales
        GROUP BY ROLLUP (year, country, category)
        """).df()
rollup_df
```

```
Out[11]:
```

	year	country	category	dollars_sold	units_sold
0	<NA>	NaN	NaN	1087931.42	38896.0
1	2025	NaN	NaN	1087931.42	38896.0
2	2025	USA	NaN	371309.46	11613.0
3	2025	Colombia	NaN	184710.89	7807.0
4	2025	Mexico	NaN	382387.89	14230.0
5	2025	Chile	NaN	149523.18	5246.0
6	2025	USA	Grocery	114985.81	3601.0
7	2025	USA	Home	64135.93	1927.0
8	2025	Colombia	Sports	30887.24	1266.0
9	2025	Mexico	Home	61481.04	2217.0
10	2025	Mexico	Grocery	122737.79	4596.0
11	2025	USA	Electronics	47506.61	1416.0
12	2025	Chile	Sports	26312.22	909.0
13	2025	Mexico	Beauty	75143.06	2952.0
14	2025	Chile	Grocery	46117.75	1632.0
15	2025	Chile	Home	25176.05	822.0
16	2025	Mexico	Electronics	51265.67	1865.0
17	2025	USA	Beauty	78809.76	2665.0
18	2025	Colombia	Electronics	21901.29	918.0
19	2025	Mexico	Sports	71760.33	2600.0
20	2025	Chile	Beauty	32357.55	1197.0
21	2025	Colombia	Home	31758.18	1284.0
22	2025	Colombia	Grocery	59981.37	2528.0
23	2025	Colombia	Beauty	40182.81	1811.0
24	2025	USA	Sports	65871.35	2004.0
25	2025	Chile	Electronics	19559.61	686.0

Conteo de ROLLUP

ROLLUP no genera todas las combinaciones como CUBE ; solo genera una ruta jerarquica siguiendo el orden definido. En este caso el orden es year , country , category , por lo que devuelve detalle por las tres columnas y subtotales cada vez mas generales.

```
In [12]: rollup_check_df = pd.DataFrame(
    {
        "metrica": ["filas_devueltas_por_rollup"],
        "valor": [len(rollup_df)],
    }
)

rollup_check_df
```

```
Out[12]:
```

	metrica	valor
0	filas_devueltas_por_rollup	26

6. Distributiva vs holística — la comparación obligatoria

GROUPING SETS con SUM

```
In [13]: groupset_sums_df = con.execute("""
    SELECT year, country, SUM(dollars_sold), SUM(units_sold)
    FROM v_sales
    GROUP BY GROUPING SETS ((year), (country), (year,country), ());
    """).df()

groupset_sums_df
```

Out[13]:

	year	country	sum(dollars_sold)	sum(units_sold)
0	2025	NaN	1087931.42	38896.0
1	<NA>	USA	371309.46	11613.0
2	<NA>	Chile	149523.18	5246.0
3	<NA>	Mexico	382387.89	14230.0
4	2025	USA	371309.46	11613.0
5	2025	Mexico	382387.89	14230.0
6	2025	Chile	149523.18	5246.0
7	2025	Colombia	184710.89	7807.0
8	<NA>	NaN	1087931.42	38896.0
9	<NA>	Colombia	184710.89	7807.0

GROUPING SETS con MEDIAN

In [14]:

```
groupset_median_df = con.execute("""
    SELECT year, country,  MEDIAN(dollars_sold), MEDIAN(units_sold)
    FROM v_sales
    GROUP BY GROUPING SETS ((year), (country), (year,country), ());
""").df()
groupset_median_df
```

Out[14]:

	year	country	median(dollars_sold)	median(units_sold)
0	<NA>	Chile	93.42	3.0
1	<NA>	USA	104.48	3.0
2	2025	NaN	91.31	3.0
3	2025	Mexico	90.17	4.0
4	2025	Chile	93.42	3.0
5	<NA>	Colombia	75.83	3.0
6	<NA>	Mexico	90.17	4.0
7	2025	USA	104.48	3.0
8	<NA>	NaN	91.31	3.0
9	2025	Colombia	75.83	3.0

Comparacion de tiempo: SUM vs MEDIAN

```
In [15]: import time

sum_query = """
SELECT year, country, SUM(dollars_sold) AS dollars_sold, SUM(units_sold) AS uni
FROM v_sales
GROUP BY GROUPING SETS ((year), (country), (year, country), ());
"""

median_query = """
SELECT year, country, MEDIAN(dollars_sold) AS dollars_sold, MEDIAN(units_sold)
FROM v_sales
GROUP BY GROUPING SETS ((year), (country), (year, country), ());
"""

def medir_consulta(query: str, repeticiones: int = 30) -> dict:
    con.execute(query).df() # corrida de calentamiento
    tiempos = []
    for _ in range(repeticiones):
        inicio = time.perf_counter()
        con.execute(query).df()
        tiempos.append(time.perf_counter() - inicio)
    return {
        "repeticiones": repeticiones,
        "tiempo_min_ms": min(tiempos) * 1000,
        "tiempo_promedio_ms": (sum(tiempos) / len(tiempos)) * 1000,
        "tiempo_max_ms": max(tiempos) * 1000,
    }

sum_timing = medir_consulta(sum_query)
median_timing = medir_consulta(median_query)

comparison_timing_df = pd.DataFrame(
    [
        {"consulta": "SUM - medida distributiva", **sum_timing},
        {"consulta": "MEDIAN - medida holistica", **median_timing},
    ]
)

comparison_timing_df
```

Out[15]:

	consulta	repeticiones	tiempo_min_ms	tiempo_promedio_ms	tiempo_max_ms
0	SUM - medida distributiva	30	5.2022	5.941873	7.0762
1	MEDIAN - medida holistica	30	3.1932	4.442830	5.7009

Respuesta ¿Cuál es más rápida? La mas rapida fue median

¿Cuál se puede particionar? ¿Por qué SUM se puede agregar incrementalmente y MEDIAN no? La que se puede particionar es la SUM, como indica la diapositiva debido a que es una

medida distributiva de computo eficiente y particionable, sum se puede sumar varias veces y al final obtener el total global pero median no ya que la mediana de varias particiones no produce la mediana global

Si tuvieras 1,000 millones de filas, ¿cuál materializarías y cuál calcularías al vuelo? ¿Por qué? seria sum debido a que como indican las diapositivas 16 y 20, la distributiva es de computo eficiente y la holistica suele aproximarse, para la materializacion total suele ocupar mucho espacio y cuesta mucho por lo que escojer la mas eficiente suena razonable.

Iceberg cube

```
In [16]: iceberg_df = con.execute("""
        select country, month, item_name, SUM(dollars_sold) AS dollars_sold
        FROM v_sales
        Group by cube (country, month, item_name)
        having SUM(dollars_sold) >= 50000
        """).df()
iceberg_df
```

```
Out[16]:
```

	country	month	item_name	dollars_sold
0	NaN	NaN	None	1087931.42
1	NaN	11	None	97816.87
2	NaN	2	None	81791.69
3	NaN	8	None	88701.09
4	NaN	4	None	91053.99
5	NaN	3	None	91386.51
6	NaN	1	None	83436.45
7	Colombia	NaN	None	184710.89
8	NaN	6	None	89586.91
9	Chile	NaN	None	149523.18
10	NaN	7	None	91463.75
11	NaN	5	None	85397.28
12	NaN	9	None	84327.79
13	NaN	10	None	91927.11
14	USA	NaN	None	371309.46
15	Mexico	NaN	None	382387.89
16	NaN	12	None	111041.98

Declaracion de uso de IA

Herramientas usadas

- ChatGPT / Codex (OpenAI) - apoyo para interpretar la actividad, revisar la estructura del proyecto, mejorar el script de generacion de datos sinteticos, explicar DuckDB y agregar celdas markdown explicativas al notebook.
- Gemini (Google) - apoyo como tutor conceptual para entender OLTP, OLAP, Data Warehouse, ETL, esquema estrella, operaciones OLAP, sintaxis SQL en DuckDB, vistas y lectura de resultados.

Que genere con IA

- ☒ Orientacion inicial para generar el dataset sintetico: prompt usado fue "Ya esta el entorno, puedes ayudarme con la generacion del dataset sintetico, el documento menciona que puedo hacerlo con 'Generas datos sinteticos de ventas con pandas + numpy.', no me generes el codigo de esto aun, ayudame a intentarlo yo, el documento de la actividad 04 que te envie menciona un ejemplo mira Generacion inicial del dataset sintetico: prompt usado fue '...' puedes ayudarme ensenando como generarlo puedes usar un ejemplo pero que no sea el de la actividad". Resumen de la ayuda: recibí una explicación conceptual, sin código completo, sobre como diseñar dimensiones y tabla de hechos, usar `np.random.default_rng(42)`, crear catálogos controlados, muestrear llaves foráneas y hacer que `dollars_sold` dependiera de `units_sold` y del precio base del producto.
- ☒ Revisión y mejora de `scripts/01_generate_data.py`: prompt usado fue "ok quiero que cheques como construí el generate data, si puedes mejorarlo hazlo, también cada vez que hagas un cambio en el código que te pida ponerlo en el md de AI_USAGE, registra los anteriores también". Resumen de la ayuda: se revisó el script y se mejoró para que el dataset cumpliera mejor la consigna: 365 días en `dim_time`, 50 productos en `dim_item`, 30 ciudades en `dim_location`, 10,000 ventas en `fact_sales`, al menos 5 categorías, más de 3 países, nombres de columnas compatibles con DuckDB y la medida `dollars_sold` calculada a partir de unidades, precio base, temporada, país y ruido lognormal.
- ☒ Explicación de `read_csv_auto` y de la distribución lognormal: prompt usado fue "puedes mostrarme como se usa read_csv_auto. también explicame el porque se menciona esto Cambie la distribución de dollars_sold de normal a lognormal porque... anade que te pregunte esto al MD". Resumen de la ayuda: se explicó como DuckDB puede leer CSVs con `read_csv_auto`, como crear tablas desde esos archivos, y por que una distribución lognormal modela mejor montos de venta que una normal.

- ☒ Correccion del dataset para alinearlo con el esquema de clase: prompt usado fue "fijate que el notebook me esta dando un error BinderException Traceback (most recent call last) Cell In[61], line 1 ... BinderException: Binder Error: table dim_item has 4 columns but 5 values were supplied; apegate a lo que venia en el data_warehousing, checa que ahi usa day como date". Resumen de la ayuda: se corrigio `scripts/01_generate_data.py` para que `dim_item.csv` exporte solo 4 columnas (`item_id`, `item_name`, `brand`, `category`) y `base_price` quedara como auxiliar interno; tambien se cambio `dim_time.csv` para que `day` fuera la fecha completa, por ejemplo `2025-01-01`.
- ☒ Explicacion de `CUBE`, `ROLLUP`, `GROUPING SETS` y DuckDB: prompt usado fue "explicame bien este codigo [bloque SQL de CUBE, ROLLUP y GROUPING SETS de la slide 24 de la presentacion] y como funciona duckdb". Resumen de la ayuda: se explico que DuckDB es un motor SQL analitico embebido, como ejecuta consultas sobre tablas y CSVs, y como `CUBE`, `ROLLUP` y `GROUPING SETS` generan distintos niveles de agregacion para analisis OLAP.
- ☒ Creacion de README y apoyo para exportar el informe: prompt usado fue "ok creame el readme, el pdf lo puedes crear tambien? pide esto PDF - informe del proyecto Notebook exportado a PDF via jupyter nbconvert --to pdf, Export to PDF en VS Code/JupyterLab, o impresion desde HTML; debe contener celdas markdown, codigo ejecutado, resultados visibles, conteos de cuboides, tiempos medidos, discusion distributiva vs holistica y declaracion de uso de IA al final". Resumen de la ayuda: se creo `README.md` con instrucciones de instalacion, generacion de datos, ejecucion del notebook, exportacion del PDF y archivos de entrega; tambien se preparo el notebook para incluir la declaracion de IA al final.
- ☒ Explicacion general del PDF `Data_Warehousing_y_Cubos_OLAP.pdf` : prompt usado fue "explicame esto porfavor.". Resumen de la ayuda: me dio un desglose estructurado del PDF, explicando la diferencia entre sistemas para operar (OLTP) y para analizar (OLAP), los tres repositorios (Base de datos, Data Warehouse, Data Lake), el proceso ETL, el modelo multidimensional (Cubo y esquema estrella), y como herramientas como DuckDB usan almacenamiento columnar para ser mas rapidas.
- ☒ Separacion OLTP/OLAP, migraciones y Data Warehousing : prompt usado fue "ok entonces son formas de usar las bases de datos oltp y olap entonces estos dos se manejan por separado y no se puede tener en una misma bd, por lo que entiendo se separa la bd para el uso de los dos, una orientada a para guardar todo y otra para analizar los datos, ahora por ejemplo, lo normal es tener las dos bases de datos y en la olap reutilizo todo de la oltp o puedo tener todo en una sola? entonces por ejemplo en que casos y como hago las migraciones de una oltp a olap o a la inversa, o si tengo alguna de las dos y quiero aplicar la otra tambien para tener las dos al mismo tiempo?, ahora explicame que es un data warehousing". Resumen de la ayuda: Gemini confirmo que tener ambas en una sola base de datos saturaria el servidor, explico que en OLAP

no se guarda todo sino lo historico y resumido de valor analitico, explico el flujo de OLTP a OLAP mediante ETL y definio Data Warehouse.

- ☒ Sintaxis SQL de operaciones OLAP con Gemini: prompt usado fue "con tu explicacion se volvio mas facil entender lo que hacen las operaciones olap, ahora el problema es programarlo, las diapositivas me mandan este codigo [bloque SQL omitido] lo que entendi del codigo fue mas o menos esto de la primera parte el dato a analizar sera sum(dollars_sold) como un total, en el cual va a incluir year, contry y category de las tablas dim_time, dim_location y dim_item y todo eso lo agrupo pero no entiendo para que, como funciona o porque asi abajo hay otro de roll up por lo visto de location establece que va estar en la misma jerarquia o los agrupa en lista los country, state, y city, pero se repite sum(dollar_sold), porque? es porque todas las jerarquias o datos se van a evaluar con respecto a una variable y en esta caso es con dollar_sold? entonces para cada dimension o tabla tengo que crear su jerarquia y asignarla con respecto a una variable". Resumen de la ayuda: Gemini explico que `t.`, `l.` e `i.` son alias SQL, que `year`, `country` y `category` son columnas de dimensiones, y que las operaciones OLAP giran alrededor de una medida numerica como `SUM(dollars_sold)`.
- ☒ JOIN, esquema estrella y agregaciones con Gemini: prompt usado fue "haber entonces actualmente tengo solo cuatro tablas desconectadas en este momento debido a que solo hice esto 'Definir el esquema estrella en SQL', para hacer las conexiones entre las tablas para formar la estrella o el cubo entonces el central es fact_sales ya que engloba las llaves foraneas y tambien tiene los datos que son las medidas numericas, entonces necesito crear las conexiones del cubo los cuales se pueden hacer de 3 formas con CUBE, ROLLUP, GROUPING SETS cube creo todas las conexiones, rollup creo solo las conexiones de una misma dimension y grouping sets las conexiones especificas que yo quiera entonces el cubo se forma de dimensiones que serian las caras del cubo y dentro de las dimensiones estan las jerarquias que son los datos ordenados, entonces rollup es para crear las jerarquias y moverse dentro de las dimensiones para obtener el dato que quieras de la jerarquia, cube y grouping sets es para crear conexiones entre dimensiones". Resumen de la ayuda: Gemini corrigio que `CUBE`, `ROLLUP` y `GROUPING SETS` no conectan tablas; las tablas se conectan con `JOIN`, y despues el `GROUP BY` calcula resúmenes.
- ☒ Cubo logico, vistas, slice & dice y pivot con Gemini: prompt usado fue "entonces la tabla de hechos siempre va a ser la que tenga las referencias o conexiones a las otras tablas y las medidas numericas, entonces el cubo lo creamos con el create, el join es solo para extraer los datos de distintas caras/dimensiones del cubo y los juntas en una nueva cara que es para ver datos, rollup es para decidir que datos de una misma cara/dimension se van a usar, cube es para generar los datos de todo el cubo ya listas, grouping sets es para generar los datos de ciertas partes del cubo, entonces por ejemplo el ano y pais con respecto a units y asi obtengos esos datos especificos de las dos caras ahora roll up y drill down para escoger el nivel de los datos de una

cara/dimension, slice & dice es para dimensiones especificas y grouping sets y cube es para lo mismo no? o esos generan los datos y slice & dice solo me los otorga o muestra? sigo sin entender como funciona pivot, me cambia la cara que estoy viendo del cubo o dado? y si o si este codigo se usa siempre para unir la tabla de hechos con las demas -- Vista que une hechos y dimensiones (el 'cubo logico') [bloque SQL omitido para no alargar el archivo]". Resumen de la ayuda: Gemini explico que `CREATE VIEW` no es obligatorio pero ayuda a no repetir `JOIN`, que slice & dice filtra partes del cubo, y que `PIVOT` rota filas a columnas para comparar resultados.

- ☒ Nivel de detalle con roll-up y drill-down con Gemini: prompt usado fue "entonces como ya cree el cubo, despues creo la forma de moverme a traves del cubo y de ahi voy generando la informacion necesaria". Resumen de la ayuda: Gemini aclaro que no se crea una forma de moverse, sino que las consultas SQL ejecutadas son el movimiento o la forma de obtener informacion.
- ☒ Consulta de roll-up por pais, anio y mes con Gemini: prompt usado fue "ok entonces no es navegar es obtener datos mas o menos especificos con.execute('"' [bloque SQL omitido para no alargar el archivo] '"') ahi le estoy haciendo una consulta roll-up preguntandole que me de los paises que mas dolares han vendido por anos y meses". Resumen de la ayuda: Gemini explico que ese bloque eran dos consultas independientes y mostro la idea de combinar ano, mes y pais en una sola consulta con `ORDER BY`.
- ☒ Roll-up/drill-down como agregar o quitar detalle con Gemini: prompt usado fue "pero entonces no es que suba o baja simplemente le dice dame los datos pero con mas o menos datos tecnicamente no?". Resumen de la ayuda: Gemini confirmo que tecnicamente roll-up y drill-down se entienden como agregar o quitar columnas en el `GROUP BY`.
- ☒ Slice & dice y pivot con dos medidas con Gemini: prompt usado fue "-- SLICE & DICE: filtrar una o varias dimensiones SELECT category, SUM(dollars_sold) FROM v_sales WHERE year = 2025 GROUP BY category; entonces aqui me esta diciendo que me va a dar el total de dolares vendidos agrupado por categorias en el ano 2025, por ejemplo lo puedo cambiar a para que en lugar de ano sea en el pais de mexico, ves que hay dos medidas numericas puedo pedir ambas o solo una de las dos? -- PIVOT: rotar ejes (sintaxis nativa de DuckDB) PIVOT v_sales ON country USING SUM(dollars_sold) GROUP BY category; aqui me va a decir el total de dolares vendidos por pais dejando los paises como columnas". Resumen de la ayuda: Gemini confirmo que se puede filtrar por pais o por ano, pedir `SUM(dollars_sold)` y `SUM(units_sold)` en la misma consulta, y usar `PIVOT` para mostrar paises como columnas agrupadas por categoria.

Que entendi y modifique yo

- El dataset debe modelarse como esquema estrella: `fact_sales` concentra las ventas y se conecta con `dim_time`, `dim_item` y `dim_location` por llaves.
- La semilla `42` permite que los CSV generados sean reproducibles.
- Entendi que `JOIN` conecta físicamente la tabla de hechos con las dimensiones, mientras que `CUBE`, `ROLLUP` y `GROUPING SETS` generan agregaciones matemáticas.
- Entendi que roll-up y drill-down no son navegación espacial, sino consultas con más o menos columnas en el `GROUP BY`.
- Entendi que `PIVOT` transforma valores de filas en columnas para facilitar reportes comparativos.
- Entendi que los valores `NULL` o `NaN` en resultados de `CUBE` representan subtotales o el total general, no errores del dataset.

Que NO supe explicar de lo que genero la IA

- No supe como contar explícitamente cuantas celdas elimina el `iceberg cube` respecto al cubo completo.
- No supe como contar explícitamente cuantas filas devuelve según la fórmula que indica elimina Materialización con `CUBE` y `ROLLUP`.
- No supe explicar lo de `read_csv_auto` por lo que pedi ayuda de la IA para que me explicara y me lo genero